



AURORA BILINGUAL SCHOOL

CLASE: PROGRAMACIÓN

Introducción a la Programación con Python

Clases 1 y 2 — Teoría básica y comandos fundamentales

Material de apoyo para el docente · Duración estimada: 2 sesiones de ~2 horas

CLASE 1 · Fundamentos y primeros pasos

Objetivo: Instalar el entorno y escribir las primeras líneas de código funcional.

Bloque	Tema	Duración
1	Introducción teórica: qué es programar, qué es Python, instalación del entorno	20–25 min
2	Primeros comandos: <code>print()</code> , comentarios, variables, tipos de datos, <code>input()</code>	30–35 min
3	Operadores básicos: aritméticos, concatenación, conversión de tipos	20 min
4	Práctica guiada en clase	20–30 min

1. Introducción teórica

- ¿Qué es un lenguaje de programación? ¿Qué es Python y por qué es tan popular hoy en día?
- Diferencia entre lenguaje **interpretado** y **compilado** (explicación breve, sin tecnicismos).
- ¿Qué es un algoritmo? Usar una analogía cotidiana, como una receta de cocina.
- Instalación del entorno: Python + editor de código (recomendado: VS Code). Alternativa sin instalación: Google Colab o Replit.

2. Primeros comandos

- **`print()`** — mostrar mensajes en pantalla.
- Comentarios en el código usando `#`.
- Variables: qué son, cómo se declaran y reglas para nombrarlas.
- Tipos de datos básicos: **`int`**, **`float`**, **`str`**, **`bool`**.
- Función **`type()`** para verificar el tipo de un dato.
- Entrada de datos con **`input()`**.

```
nombre = input("¿Cómo te llamas? ")
edad = int(input("¿Cuántos años tienes? "))
print("Hola", nombre, "- tienes", edad, "años")
```

3. Operadores básicos

- Operadores aritméticos: `+`, `-`, `*`, `/`, `//`, `%`, `**`
- Concatenación de cadenas de texto (strings).
- Conversión de tipos: **`int()`**, **`float()`**, **`str()`**.

```
a = 7
b = 2
print(a / b) # división normal -> 3.5
print(a // b) # división entera -> 3
print(a % b) # resto/módulo -> 1
print(a ** b) # potencia -> 49
```

4. Práctica guiada en clase

- Ejercicio 1: crear un programa que pida el nombre y la edad del usuario, y lo salude por pantalla.



- Ejercicio 2: construir una calculadora simple que sume, reste, multiplique y divida dos números.

■ Tarea para casa

Resolver 3 o 4 ejercicios cortos de práctica sobre variables, tipos de datos y operadores (se recomienda entregar una guía impresa o digital aparte con los enunciados).

CLASE 2 · Lógica y control de flujo

Objetivo: Aprender a tomar decisiones y repetir acciones dentro del código.

Bloque	Tema	Duración
1	Repaso de la clase anterior y resolución de dudas	10 min
2	Condicionales: comparación, operadores lógicos, if / elif / else	30 min
3	Bucles: while, for, range(), break y continue	35–40 min
4	Práctica integradora	25–30 min

1. Repaso rápido

- Revisión de la tarea asignada en la Clase 1.
- Espacio abierto para resolver dudas sobre variables, tipos de datos y operadores.

2. Condicionales

- Operadores de comparación: `==` `!=` `>` `<` `>=` `<=`
- Operadores lógicos: **and**, **or**, **not**.
- Estructura condicional: **if**, **elif**, **else**.
- Ejemplo aplicado: clasificar un número como positivo, negativo o cero.

```
numero = int(input("Escribe un número: "))

if numero > 0:
    print("Es positivo")
elif numero < 0:
    print("Es negativo")
else:
    print("Es cero")
```

3. Bucles

- **while**: cuándo conviene usarlo y el riesgo de generar bucles infinitos.
- **for** combinado con **range()**.
- Introducción breve a **break** y **continue**.
- Ejemplo: tabla de multiplicar usando for.
- Ejemplo: contador usando while.

```
# Tabla de multiplicar del 5
for i in range(1, 11):
    print(5, "x", i, "=", 5 * i)
```

4. Práctica integradora

- Ejercicio 1: validar si un número ingresado es par o impar.
- Ejercicio 2: juego 'adivina el número' usando while y condicionales.



- Mini-reto: sumar todos los números del 1 al 100 utilizando un bucle.

■ Tarea para casa

Resolver un ejercicio que combine condicionales y bucles, por ejemplo: construir un menú simple de opciones (usando un bucle while) que permita al usuario elegir distintas acciones hasta que decida salir.

Recomendaciones generales para el docente

- **Ritmo:** dejar tiempo extra para preguntas; en la primera clase, la instalación del entorno suele tomar más tiempo del previsto.
- **Ejemplos cotidianos:** anclar conceptos abstractos a situaciones reales (un semáforo para condicionales, una receta para algoritmos).
- **Código en vivo:** escribir el código junto a los estudiantes en lugar de solo mostrar diapositivas; genera más participación y permite detectar errores comunes en el momento.
- **Errores como aprendizaje:** cuando aparezca un error, resolverlo en voz alta explicando el mensaje; esto ayuda a desmitificar el miedo al error desde el inicio.